МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 6. Реализация численных алгоритмов

Студент
гр. БИН-26-1 _____ Р.Е. Гаража

Преподаватель _____ М.В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Наибольший общий делитель

Пользователь вводит два целых числа a и b на отдельных строках. Если хотя бы одно число отрицательное или оба числа равны нулю, вывести Ошибка. Для остальных значений найти наибольший общий делитель алгоритмом Евклида.

Алгоритм: проверить входные данные на корректность. Если a < 0, b < 0 или (a = 0 и b = 0), вывести «Ошибка». Иначе выполнять цикл, пока b > 0: заменить пару (a, b) на (b, a mod b). Первое число полученной пары является результатом.

Изображение схемы для решения первого задания 1.

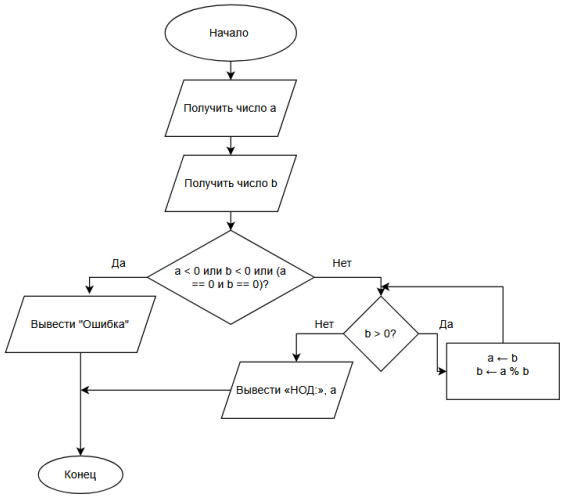


Рисунок 1 – Схема программы для первого задания

Изображение результата выполнения программы 2.

lab6 > lab1.1.py > ...

```
1   a = int(input())
2   b = int(input())
3
4   if a < 0 or b < 0 or (a == 0 and b == 0):
5       print("Ошибка")
6   else:
7       while b > 0:
8           a, b = b, a % b
9       print("НОД:", a)
10
```

Рисунок 2 – Код первого задания

Падание демонстрирует реализацию алгоритма Евклида для нахождения НОД. Ключевой момент — проверка входных данных и последовательная замена пары чисел до получения нуля во втором элементе.

1.2 Задание 2. Проверка числа на простоту

Пользователь вводит целое число n. Вывести Простое или Не простое. Все значения меньше 2 дают результат Не простое. Проверку завершить при нахождении делителя или когда квадрат проверяемого делителя превысит n.

Изображение схемы для решения второго задания 3.

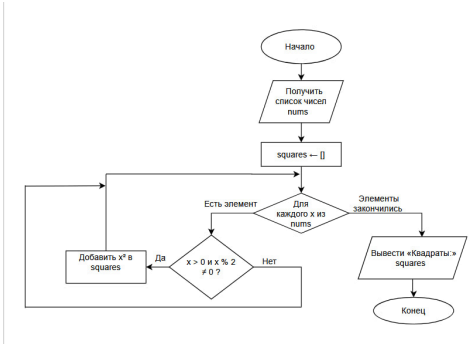


Рисунок 3 – Схема программы для второго задания

Алгоритм: если n < 2, вывести «Не простое». Иначе запустить цикл с делителем i от 2 до тех пор, пока i * i <= n. Если n делится на i без остатка, число не простое. Если цикл завершился без нахождения делителей, число простое.

Изображение результата выполнения программы 4.

lab6 > lab1.2.py > ...

```python
n = int(input())

if n < 2:
    print("Не простое")
else:
    is_prime = True
    d = 2
    while d * d <= n:
        if n % d == 0:
            is_prime = False
            break
        d += 1

    if is_prime:
        print("Простое")
    else:
        print("Не простое")

```

Рисунок 4 – Код второго задания

Задание тренирует использование цикла с условием для проверки делителей. Важно ограничить перебор квадратным корнем из числа для оптимизации алгоритма.

1.3 Задание 3. Разложение на простые множители

Пользователь вводит целое число n. Если n < 2, вывести Ошибка. Разложить допустимое число на простые множители и вывести их в порядке неубывания на одной строке через один пробел, без пробела в конце.

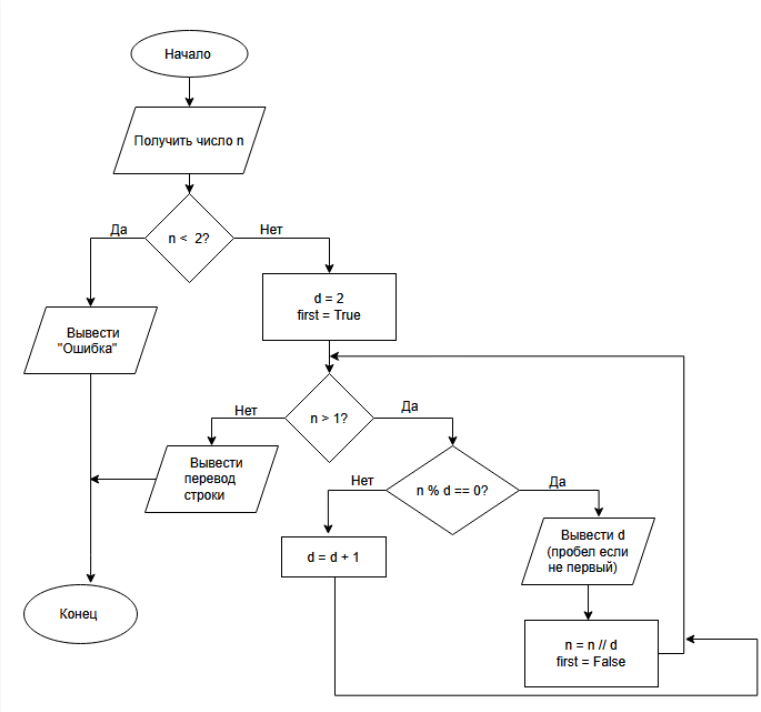


Рисунок 5 – Схема программы для третьего задания

Алгоритм: проверить n < 2. Установить делитель i = 2. Пока i * i <= n, проверять деление без остатка. При делении выводить i и уменьшать n. После цикла, если n > 1, вывести оставшееся n.

код программы 6.

lab6 > lab1.3.py > ...

```python
n = int(input())

if n < 2:
    print("Ошибка")
else:
    d = 2
    first = True
    while n > 1:
        if n % d == 0:
            if not first:
                print(" ", end="")
            print(d, end="")
            n //= d
            first = False
        else:
            d += 1
    print()
```

Рисунок 6 – Код третьего задания

Задание показывает обработку последовательности с вложенными циклами. Основная задача — повторно проверять один и тот же делитель при успешном делении до полного разложения числа.

1.4 Задание 4. Цифры числа

Пользователь вводит целое число n. Если число отрицательное, вывести Ошибка. Определить количество цифр и их сумму, последовательно выделяя последнюю цифру остатком от деления на 10 и удаляя ее целочисленным делением на 10. У числа 0 одна цифра и сумма цифр равна 0.

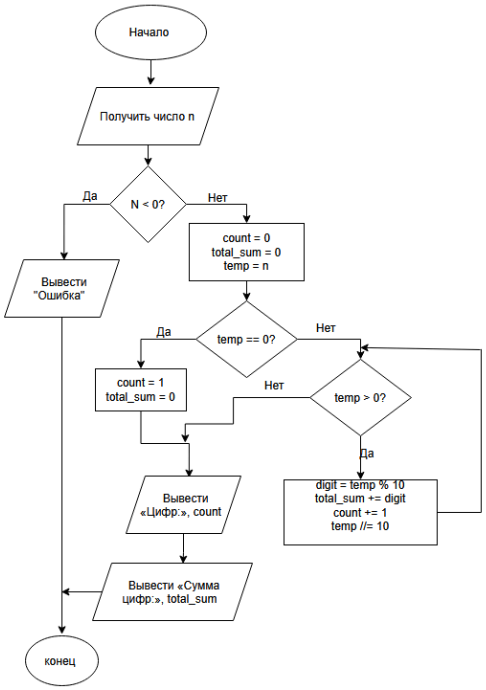


Рисунок 7 – Схема программы для четвертого задания

Алгоритм: проверить на отрицательность. Инициализировать счетчик цифр и сумму нулями. Если n = 0, установить счетчик в 1. Иначе в цикле, пока n > 0, добавлять n % 10 к сумме, увеличивать счетчик на 1 и выполнять n

код программы 8.

```
n = int(input())

if n < 0:
    print("Ошибка")
else:
    count = 0
    total_sum = 0
    temp = n

    if temp == 0:
        count = 1
        total_sum = 0
    else:
        while temp > 0:
            digit = temp % 10
            total_sum += digit
            count += 1
            temp //= 10

    print("Цифр:", count)
    print("Сумма цифр:", total_sum)

```

Рисунок 8 – Код четвертого задания

Задание демонстрирует извлечение цифр числа с помощью арифметических операций без преобразования в строку. Отдельно обрабатывается краевой случай с числом 0.

1.5 Задание 5. Таблица расстояний

Пользователь вводит целое число n от 1 до 10 включительно. При выходе за этот диапазон вывести Ошибка. Построить таблицу расстояний между каждой парой точек на прямой с координатами от 0 до n - 1.

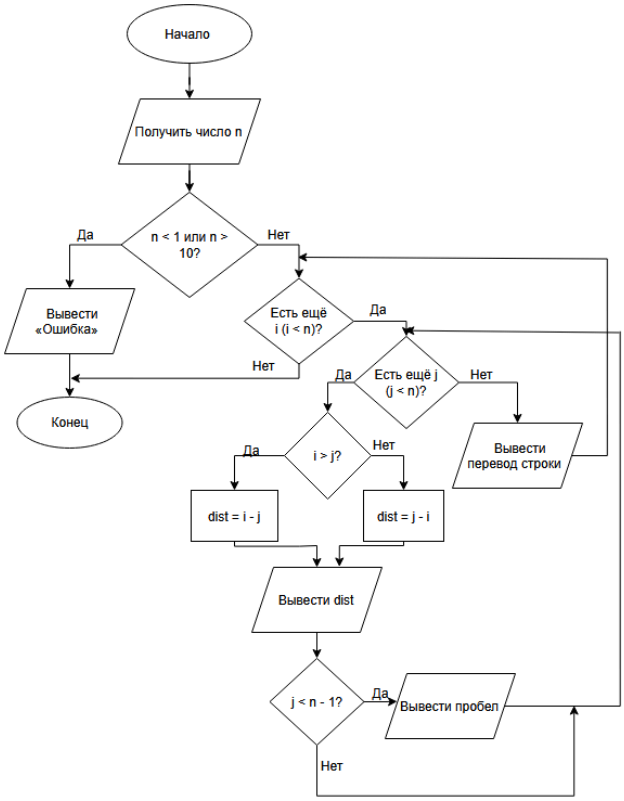


Рисунок 9 – Схема программы для пятого задания

Алгоритм: проверить диапазон введенного n. Использовать вложенные циклы: внешний для строк i от 0 до n-1, внутренний для столбцов j от 0 до n-1. Вычислять расстояние как модуль разности |i - j|. Элементы строки разделять одним пробелом.

код программы 10.

lab6 > lab1.5.py > ...

```python
n = int(input())

if n < 1 or n > 10:
    print("Ошибка")
else:
    for i in range(n):
        for j in range(n):
            if i > j:
                dist = i - j
            else:
                dist = j - i

            print(dist, end="")
            if j < n - 1:
                print(" ", end="")
        print()
```

Рисунок 10 – Код пятого задания

Задание учит работе с вложенными циклами и формированию матрицы значений. Важно аккуратно сформировать строку вывода без лишних пробелов в конце.